TP RSA Sécurité et Pratiques

Arthur Walter · Master 2 Data Science

Sécurité des systèmes d'information · Abdeljalil ABBAS-TURKI · 16 octobre 2026

Partie 1 Implémentation de base du RSA

Question 1a Compléter l'exponentiation modulaire rapide

La fonction home_mod_expnoent(x, y, n) calcule x puissance y modulo n. Elle évite de construire toute la puissance : elle utilise des carrés successifs et ne multiplie au résultat que les puissances nécessaires. Pour cela, l'exposant est lu en binaire, de droite à gauche.

À chaque multiplication, on ne garde que le reste de la division par n. Par exemple, 17 modulo 5 vaut 2, puisque 17 = 3 × 5 + 2. De même, 2 puissance 3 modulo 5 donne 3 : on calcule 8, puis son reste dans la division par 5. Le nombre de tours de la boucle correspond à la longueur de l'exposant en bits.

Pour x = 3, y = 13 et n = 17, la fonction retourne 12. Comme 13 s'écrit 1101 en binaire, le calcul utilise trois multiplications conditionnelles. Les tests comparent également 300 cas à pow(x, y, n), utilisé comme référence de vérification.

Question 1b Compléter l'algorithme d'Euclide étendu

home_ext_euclide(y, b) cherche le nombre qui, multiplié par b, donne un reste de 1 modulo y. C'est ce que l'on appelle un inverse modulaire. Le module vient en premier dans l'appel ; le calcul de la clé privée d'Alice est donc home_ext_euclide(lambda_a, ea).

L'algorithme suit les divisions d'Euclide et le coefficient de b dans l'identité de Bézout. Si le PGCD vaut 1, ce coefficient fournit l'inverse recherché.

$$s \times y + t \times b = 1$$

$$t \times b \equiv 1 \pmod{y}$$

Le résultat est ramené dans l'intervalle de 0 à y − 1. Lorsque le PGCD est différent de 1, la fonction signale qu'aucun inverse n'existe.

Avec le module 20 et le nombre 3, l'inverse vaut 7 : 3 × 7 = 21, qui laisse un reste de 1 modulo 20. La même méthode donne 2753 comme inverse de 17 modulo 3120 :

$$17 \times 2753 = 15 \times 3120 + 1$$

Le calcul du squelette nommé phia ou phib utilise un PPCM. Il correspond donc à λ(n), et non à l'indicatrice d'Euler φ(n). Les variables ont été renommées lambda_a et lambda_b. Dans les deux paires de clés, le produit de l'exposant public et de l'exposant privé vaut bien 1 modulo λ(n).

Partie 1 Échange et remplacement de MD5

Question 2 Simuler l'échange entre Bob et Alice

Bob commence par représenter son texte avec un nombre. Le texte passe en octets UTF-8, puis en entier big-endian. Il chiffre cet entier avec la clé publique d'Alice, formée de na et ea. Alice utilise ensuite da, sa clé privée, pour retrouver l'entier et reconstruire le texte.

Pour la signature, ce sont les clés de Bob qui interviennent. Bob signe l'empreinte du texte avec db. Alice recalcule l'empreinte du message qu'elle a retrouvé et vérifie les signatures avec eb et nb. Le chiffrement sert donc à réserver la lecture à Alice ; la signature permet de vérifier le contenu avec la clé de Bob, à condition de disposer de sa véritable clé publique.

Contrôle sur Bonjour Alice	Résultat
Longueur UTF-8	13 octets
Entier du message	526354504649867183149086837232 5
Message après déchiffrement	Bonjour Alice
Vérification de la signature	Valide

La longueur en octets accompagne l'envoi pour ne pas perdre les éventuels zéros au début. Les deux conversions ont été modifiées ensemble pour conserver aussi les accents. Le détail des valeurs calculées se trouve dans resultats/execution_complete.json.

Question 3a Remplacer MD5 par SHA 256

L'empreinte est calculée par hashlib.sha256(message.encode("utf-8")).digest(). Le passage intermédiaire par binascii.b2a_uu a été retiré. SHA-256 produit 32 octets, contre 16 pour MD5.

Le module de Bob ne fait que 206 bits : une empreinte de 256 bits ne tient pas nécessairement dans une seule opération RSA. Pour garder les clés fournies sans réduire l'empreinte modulo nb, elle est divisée en deux blocs de 25 et 7 octets, signés séparément. Alice doit valider les deux blocs. Il s'agit d'un choix d'adaptation au squelette, pas d'une signature RSA standard.

Question 3b Déterminer la nouvelle limite du message

La contrainte du chiffrement reste m < na. Pour accepter tous les contenus de L octets, il faut que 256 puissance L − 1 soit inférieur à na. Le module d'Alice fait 203 bits : 25 octets, soit 200 bits, tiennent toujours ; 26 octets peuvent dépasser cette taille.

La limite retenue est donc de 25 octets UTF-8. Cela permet 25 caractères ASCII, mais seulement 12 caractères é suivis d'un caractère ASCII. Certains entiers codés sur 26 octets pourraient tenir, mais la règle uniforme les refuse.

Cette augmentation par rapport aux 10 caractères du squelette vient de la correction de la limite de saisie. Le remplacement de MD5 par SHA-256 ne change pas la capacité du module RSA. Les messages plus longs sont traités par blocs dans la partie 3.

Partie 2 Simulation du canal auxiliaire

Question 1 Compter et interpréter les multiplications conditionnelles

home_mod_expnoent_trace fait le même calcul qu'en partie 1, mais garde une trace des opérations. Chaque bit à 1 entraîne une multiplication supplémentaire. La fonction compte ces multiplications et note l'ordre des bits rencontrés ; elle compte les carrés à part.

Pour 3 puissance 13 modulo 17, le résultat est 12. La fonction compte trois multiplications conditionnelles et quatre carrés. Les bits sont rencontrés dans l'ordre 1, 0, 1, 1 ; leur lecture dans l'autre sens donne la représentation habituelle 1101.

Le nombre total de multiplications conditionnelles donne le poids de Hamming de l'exposant, c'est-à-dire son nombre de bits à 1. Il ne permet pas, à lui seul, de connaître leur position. Par exemple, les exposants 11 et 13 comportent chacun trois bits à 1 sur quatre bits.

Un observateur qui distingue les opérations de chaque tour peut en déduire davantage que ce simple total : la position des multiplications peut révéler les bits de l'exposant. Dans le TP, ces événements sont notés directement par le programme. Ils représentent ce que l'on pourrait chercher à observer ; aucune mesure physique de consommation ou de cache n'a été réalisée.

Question 2 Bonus Comparer plusieurs messages

Les essais sur les entiers 1, 42, 513 et 123456789 donnent tous 85 multiplications conditionnelles et 198 carrés pour le déchiffrement direct avec la clé d'Alice. Ces nombres dépendent de l'exposant privé, qui ne change pas entre les messages.

Les temps sont mesurés avec perf_counter_ns sur cinq séries de trente calculs. Le tableau donne la médiane du temps moyen par opération, avec les paramètres CRT déjà préparés. Les mesures proviennent de l'exécution enregistrée sous Python 3.12.14 sur Linux.

Message entier	Direct en ns	CRT en ns
1	62661	43763
42	361520	182812
513	267391	246892
123456789	156573	112352

Le CRT est plus rapide dans cette série de mesures. Les durées restent sensibles à la machine, à sa charge et aux valeurs manipulées. Le cas du message 1 est notamment moins coûteux. Ces variations ne suffisent pas à déduire la clé privée à partir d'un simple chronométrage.

Partie 2 Optimisation par le CRT

Question 3a Calculer les exposants réduits

Au lieu de faire tout le déchiffrement modulo n, on fait deux calculs, l'un modulo p et l'autre modulo q. Ces nombres sont plus petits. Les exposants réduits sont préparés une fois, puis réutilisés :

$$dp = d \mod (p - 1)$$

$$dq = d \mod (q - 1)$$

L'inverse de q modulo p, noté q_inv, est obtenu avec la fonction d'Euclide étendu.

Question 3b Reconstituer le message

Les résultats mp et mq ne sont pas deux morceaux du texte. Ce sont deux restes du même message : l'un modulo p, l'autre modulo q. Le CRT permet de retrouver le seul entier entre 0 et n − 1 qui respecte ces deux restes :

$$mp = c^{dp} \mod p$$

$$mq = c^{dq} \mod q$$

$$h = ((mp - mq) \times q_inv) \mod p$$

$$m = (mq + h \times q) \mod n$$

Dans l'exemple du TP, p = 137, q = 131, c = 8363 et d = 11787. Le programme obtient dp = 91, dq = 87 et q_inv = 114. Les résultats intermédiaires sont mp = 102 et mq = 120.

On trouve alors h = 3, puis m = 120 + 3 × 131 = 513. Le rechiffrement avec e = 3 redonne 8363. Le calcul CRT et le calcul direct restituent donc le même message.

Les tests couvrent également les deux clés du TP, avec des chiffrés pseudoaléatoires et les valeurs limites 0, 1, p, q et n − 1. Le cas d'un chiffré divisible par p ou q est traité explicitement, y compris lorsqu'un exposant réduit est nul.

Question 3c Bonus Commenter le lien avec la surface d'attaque

Le CRT diminue le coût du calcul en travaillant sur des modules plus petits. Cela ne supprime pas les canaux auxiliaires : les calculs utilisent encore des exposants secrets, dp et dq. Une durée d'exécution plus courte ne suffit donc pas à conclure à une réduction générale de la surface d'attaque.

Le CRT peut aussi être sensible aux attaques par faute. Dans certains scénarios, un résultat erroné sur l'une des deux branches peut aider à retrouver un facteur du module. Des protections telles que l'aveuglement, la vérification du résultat et des primitives à temps constant restent nécessaires dans une implémentation réelle.

Partie 3 Bourrage et traitement par blocs

Question 1 Découper chiffrer et reconstituer le message

Le module d'Alice occupe 203 bits, soit k = 26 octets après arrondi. Le sujet impose j inférieur ou égal à k/2 ; chaque morceau du message contient donc au maximum 13 octets.

Le texte est encodé en UTF-8 puis découpé en octets. Pour chaque morceau, le programme génère k − j − 3 octets non nuls, regroupés dans PS, et construit le bloc suivant :

$$EM = 00 \;||\; 02 \;||\; PS \;||\; 00 \;||\; morceau$$

Élément du bloc plein	Taille
Préfixe 00 02	2 octets
PS non nul	10 octets
Séparateur 00	1 octet
Morceau du message	13 octets
Bloc complet	26 octets

Les valeurs de PS sont tirées avec secrets.randbelow(255) + 1, donc entre 1 et 255, comme le domaine demandé par alea % 255 + 1. La longueur de PS est toujours d'au moins huit octets. Si le dernier morceau est plus court, PS est allongé pour conserver la taille du bloc.

Le bloc est converti en entier et chiffré avec la clé publique d'Alice. Son premier octet nul garantit ici une valeur inférieure au module. Au déchiffrement, to_bytes reconstruit exactement 26 octets afin de retrouver ce zéro initial.

Alice vérifie le préfixe, le séparateur et les tailles, puis enlève le bourrage. Elle réunit d'abord les octets et les décode seulement après, car un caractère UTF-8 peut être coupé entre deux blocs. La signature est vérifiée sur le texte complet retrouvé.

Le texte d'essai « Bonjour Alice ! Ce message dépasse vingt-cinq octets et contient des accents é et un emoji 😀. » occupe 98 octets et produit huit blocs. Le texte retrouvé est identique et la signature est valide. Le déchiffrement direct donne le même résultat que le CRT.

Le bourrage rend le chiffrement aléatoire : deux envois du même message donnent normalement des chiffrés différents. Ce format reste celui du TP, inspiré de PKCS#1 v1.5. Un service révélant la validité du bourrage pourrait servir de padding oracle. Le message d'erreur générique utilisé ici ne garantit pas un temps d'exécution constant.

Partie 3 Certificat du site Springer

Question 2a Identifier la clé publique

Le certificat présenté par link.springer.com a été relevé le 16 octobre 2026 à 16:02:25 UTC, avec vérification de la chaîne et du nom d'hôte. Son sujet est CN=*.springer.com. Il contient une clé publique RSA de 2048 bits, avec l'exposant 65537, soit 0x10001.

Le relevé provient du job GitHub Actions 117066939109. Le certificat public est conservé dans resultats/certificat_springer.pem et les champs extraits dans certificat_springer.json.

Question 2b Identifier la fonction d'empreinte

L'algorithme de signature du certificat est sha256WithRSAEncryption, d'OID 1.2.840.113549.1.1.11. Le hash utilisé dans cette signature est donc SHA-256.

L'empreinte SHA-256 du certificat complet, calculée séparément pour identifier le fichier observé, vaut :

`27d5c26b2047ca2b459030694a9fed4e68157c22b802be2ffb231ee09d327350`

Question 2c Identifier le bourrage

La signature du certificat utilise RSASSA-PKCS1-v1_5. Cet encodage concerne la signature produite par l'autorité émettrice. Ce n'est pas le bloc de chiffrement 00 02 utilisé dans la partie précédente.

Autre champ observé	Valeur
Autorité émettrice	Let's Encrypt, YR2
Début de validité UTC	23 septembre 2026 à 11:18:57
Fin de validité UTC	22 décembre 2026 à 11:18:56
Version TLS négociée	TLS 1.3
Suite négociée	TLS_AES_128_GCM_SHA256

La présence d'une clé RSA dans le certificat ne signifie pas que le trafic est chiffré directement par RSA. La connexion observée utilise AES-GCM pour protéger les données et TLS 1.3 ne reprend pas l'ancien échange de clés RSA. Le schéma de signature de CertificateVerify ne peut pas être déduit du seul certificat.

Les champs du JSON ont été comparés à ceux du PEM conservé. Ces valeurs décrivent le certificat présenté au moment du relevé ; elles pourront changer lors d'un renouvellement ou selon la négociation du client.

Vérifications avec de petits exemples

Les exemples ci-dessous utilisent les mêmes fonctions que le TP. Les petites valeurs servent uniquement à suivre les opérations à la main. Les résultats sont aussi enregistrés dans resultats/exemples_simples.json.

Retrouver un message avec de petites clés

On prend p = 11 et q = 5, donc n = 55 et λ(n) = 20. On choisit e = 3 et d = 7 : ces exposants vont ensemble, car 3 × 7 laisse un reste de 1 modulo 20. Pour ce seul exemple, le message est directement l'entier 8, sans texte ni bourrage.

$$\textit{Chiffrement} \ : \ 8^{3} \ \textit{mod} \ 55 \ = \ 17$$

$$\textit{Déchiffrement} \ : \ 17^{7} \ \textit{mod} \ 55 \ = \ 8$$

On retrouve donc bien 8. Dans le programme principal, le principe est le même, mais le texte est d'abord converti en entier et les clés sont celles de l'énoncé.

Vérifier le même calcul avec le CRT

Pour ce chiffré 17, on trouve dp = 7, dq = 3 et q_inv = 9. Les deux calculs donnent mp = 8 et mq = 3. La recombinaison donne h = 1, puis m = 3 + 1 × 5 = 8. Le calcul direct et le CRT arrivent bien au même résultat.

Changer une seule lettre dans le message

Les textes Bonjour et bonjour ne diffèrent que par la majuscule. Leurs empreintes SHA-256 sont pourtant différentes. Voici leurs seize premiers caractères hexadécimaux ; les empreintes complètes restent de 32 octets.

Texte	Début du SHA-256
Bonjour	9172e8eec99f144f
bonjour	2cb4b1431b84ec15

Vérifier le découpage en blocs

Les 26 lettres ABCDEFGHIJKLMNOPQRSTUVWXYZ occupent 26 octets en UTF-8. Avec 13 octets par morceau, on obtient deux blocs : ABCDEFGHIJKLM puis NOPQRSTUVWXYZ. Après déchiffrement et retrait du bourrage, leur réunion restitue l'alphabet complet.

Un accent illustre pourquoi le décodage attend la fin : douze lettres A suivies de é occupent 14 octets. Les deux octets de é, C3 et A9 en hexadécimal, se retrouvent de part et d'autre de la coupure à 13 octets. Réunir les octets avant de décoder permet de retrouver la lettre.

Exécution et vérifications

Les fichiers Walter_Arthur_main.py et Walter_Arthur_util.py doivent rester dans le même dossier. Le premier exécute les parties du TP ; le second contient les fonctions de calcul. Depuis ce dossier, les commandes utilisées sont :

```
python Walter_Arthur_main.py --test
python Walter_Arthur_main.py --partie toutes --message "Bonjour Alice"
python Walter_Arthur_main.py --partie 3 --message "Un message long avec accents"
```

L'option --test utilise le dossier tests du dépôt. Les calculs RSA ne demandent aucune bibliothèque externe. L'option toutes inclut la partie 1 et reste limitée à 25 octets UTF-8 ; la partie 3 accepte les messages découpés sur plusieurs blocs.

Les quinze tests RSA passent. Ils contrôlent l'exponentiation, les inverses, les conversions, les signatures, les traces, le CRT et le bourrage. Les cas comprennent les messages vides, les zéros initiaux, les accents, les emoji et plusieurs altérations de l'envoi. Les mêmes tests ont réussi sous Python 3.10 et 3.13 dans la CI.

Trois tests distincts ont aussi vérifié le lecteur de certificats sur des certificats locaux de test : RSA, EC et clé EC avec signature RSA de l'émetteur. Ils valident l'extraction des champs ; les valeurs Springer proviennent du certificat réellement relevé, et non de ces jeux de test.

Les résultats du TP sont conservés dans execution_complete.json et message_long.json. Les tests vérifient le comportement du programme, sans établir la sécurité de l'échange. Les clés restent petites, la signature SHA-256 en deux blocs est non standard et les calculs Python ne sont pas à temps constant. Le remplacement de MD5 et l'ajout du bourrage ne suffisent pas à rendre cette construction adaptée à un usage réel.